

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

NAME: BP Venkata Uday Kumar

COURSE CODE: CSA-0409

REG NO: 192411203

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

Experiment Questions:

1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.

ANS:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int base,limit,logical,physical;
```

```
    printf("Enter Base Register: ");
```

```
    scanf("%d",&base);
```

```
    printf("Enter Limit Register: ");
```

```
    scanf("%d",&limit);
```

```
    printf("Enter Logical Address: ");
```

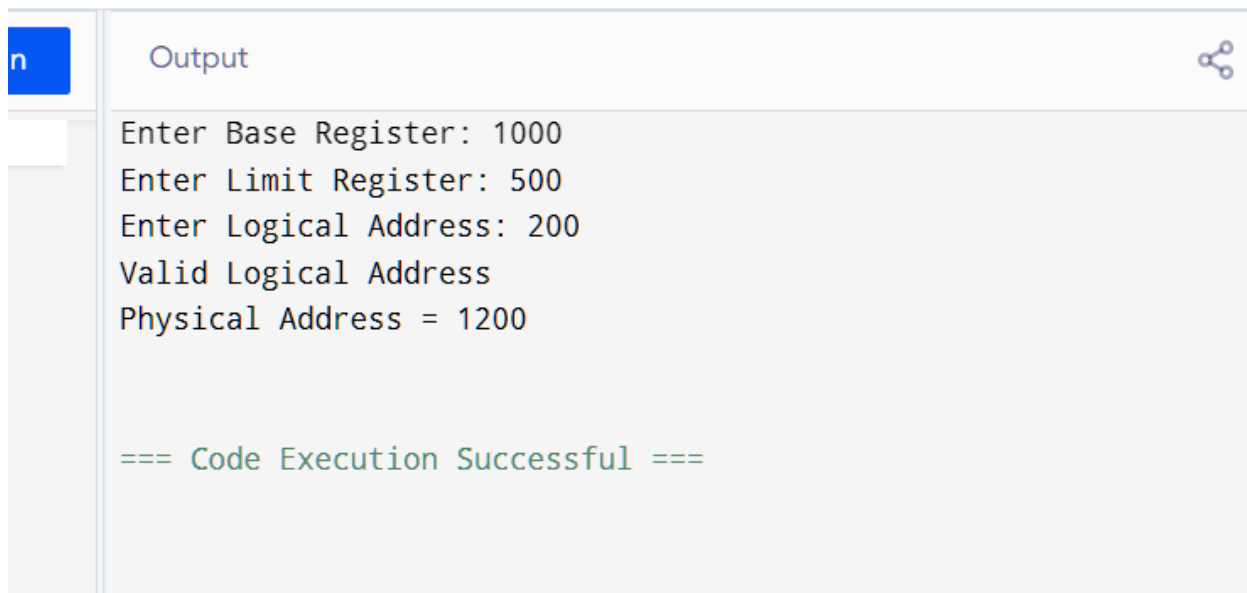
```
    scanf("%d",&logical);
```

```

if(logical>=0 && logical<limit)
{
    physical=base+logical;
    printf("Valid Logical Address\n");
    printf("Physical Address = %d\n",physical);
}
else
{
    printf("Invalid Logical Address\n");
    printf("Address is outside the limit.\n");
}

return 0;
}

```



```

n      Output
Enter Base Register: 1000
Enter Limit Register: 500
Enter Logical Address: 200
Valid Logical Address
Physical Address = 1200

=== Code Execution Successful ===

```

2. Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int pageSize,numPages,page,offset;  
int pageTable[100];  
int frame,physical;
```

```
printf("Enter Page Size: ");  
scanf("%d",&pageSize);
```

```
printf("Enter Number of Pages: ");  
scanf("%d",&numPages);
```

```
printf("Enter Page Table:\n");
```

```
for(int i=0;i<numPages;i++)  
{  
    printf("Page %d -> Frame: ",i);  
    scanf("%d",&pageTable[i]);  
}
```

```
printf("Enter Page Number: ");  
scanf("%d",&page);
```

```
printf("Enter Offset: ");  
scanf("%d",&offset);
```

```
if(page<0 || page>=numPages)  
{  
    printf("Invalid Page Number\n");  
}  
else if(offset<0 || offset>=pageSize)  
{  
    printf("Invalid Offset\n");  
}  
else
```

```

{
    frame=pageTable[page];
    physical=frame*pageSize+offset;

    printf("Page Number = %d\n",page);
    printf("Frame Number = %d\n",frame);
    printf("Physical Address = %d\n",physical);
}

return 0;
}

```

3.Develop a program to simulate a page table

```

Output
Enter Page Size: 100
Enter Number of Pages: 4
Enter Page Table:
Page 0 -> Frame: 3
Page 1 -> Frame: 1
Page 2 -> Frame: 3
Page 3 -> Frame: 2
Enter Page Number: 3
Enter Offset: 3
Page Number = 3
Frame Number = 2
Physical Address = 203

=== Code Execution Successful ===

```

3.The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.
#include<stdio.h>

```

int main()
{
    int pageTable[100];
    int n,page,offset,frame,physical;

    printf("Enter Number of Pages: ");
    scanf("%d",&n);

    printf("Enter Page Table Entries:\n");

```

```
for(int i=0;i<n;i++)
{
    printf("Page %d -> Frame: ",i);
    scanf("%d",&pageTable[i]);
}

printf("\nEnter Page Number: ");
scanf("%d",&page);

printf("Enter Offset: ");
scanf("%d",&offset);

if(page<0 || page>=n)
{
    printf("Invalid Page Number\n");
}
else
{
    frame=pageTable[page];

    printf("\nPage Number = %d\n",page);
    printf("Frame Number = %d\n",frame);

    physical=frame*100+offset;

    printf("Physical Address = %d\n",physical);
}

return 0;
}
```

```
Run      Output
Enter Number of Pages: 5
Enter Page Table Entries:
Page 0 -> Frame: 3
Page 1 -> Frame: 2
Page 2 -> Frame: 2
Page 3 -> Frame: 4
Page 4 -> Frame: 6

Enter Page Number: 2
Enter Offset: 4

Page Number = 2
Frame Number = 2
Physical Address = 204

=== Code Execution Successful ===
```

4. Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.

```
#include<stdio.h>

int main()
{
    float memoryTime,tlbTime,hitRatio;
    float withoutTLB,withTLB;

    printf("Enter Memory Access Time: ");
    scanf("%f",&memoryTime);

    printf("Enter TLB Access Time: ");
    scanf("%f",&tlbTime);

    printf("Enter TLB Hit Ratio: ");
    scanf("%f",&hitRatio);

    withoutTLB=2*memoryTime;

    withTLB=hitRatio*(tlbTime+memoryTime)
        +(1-hitRatio)*(tlbTime+2*memoryTime);
```

```

printf("\nEffective Access Time Without TLB = %.2f\n",withoutTLB);
printf("Effective Access Time With TLB = %.2f\n",withTLB);

if(withTLB<withoutTLB)
    printf("TLB improves memory access time.\n");
else
    printf("TLB does not improve memory access time.\n");

return 0;
}

```

```

Output

Enter Memory Access Time: 100
Enter TLB Access Time: 10
Enter TLB Hit Ratio: 0.8

Effective Access Time Without TLB = 200.00
Effective Access Time With TLB = 130.00
TLB improves memory access time.

=== Code Execution Successful ===

```

5. Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

```
#include<stdio.h>
```

```

int main()
{
    int base,limit,logical,physical;

```

```
printf("Enter Base Register: ");
scanf("%d",&base);

printf("Enter Limit Register: ");
scanf("%d",&limit);

printf("Enter Logical Address: ");
scanf("%d",&logical);

if(logical>=0 && logical<limit)
{
    physical=base+logical;

    printf("\nAccess Allowed\n");
    printf("Logical Address = %d\n",logical);
    printf("Physical Address = %d\n",physical);
}
else
{
    printf("\nAccess Denied\n");
    printf("Unauthorized Memory Access!\n");
}

return 0;
}
```



```

Output

Enter Base Register: 5000
Enter Limit Register: 1000
Enter Logical Address: 120

Access Allowed
Logical Address = 120
Physical Address = 5120

=== Code Execution Successful ===

```

RUBRICS FOR CALCULATION:

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand